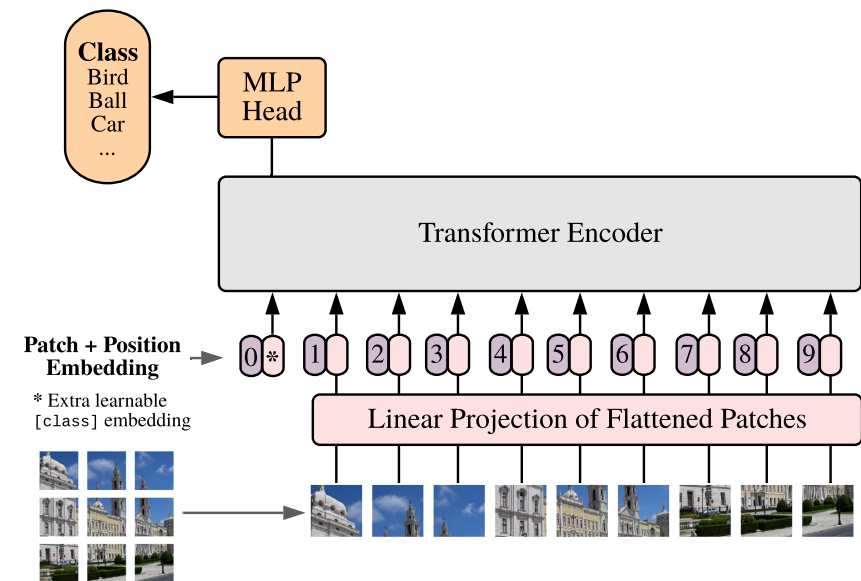


Vision Transformer



27. März 2025

The attention mechanism

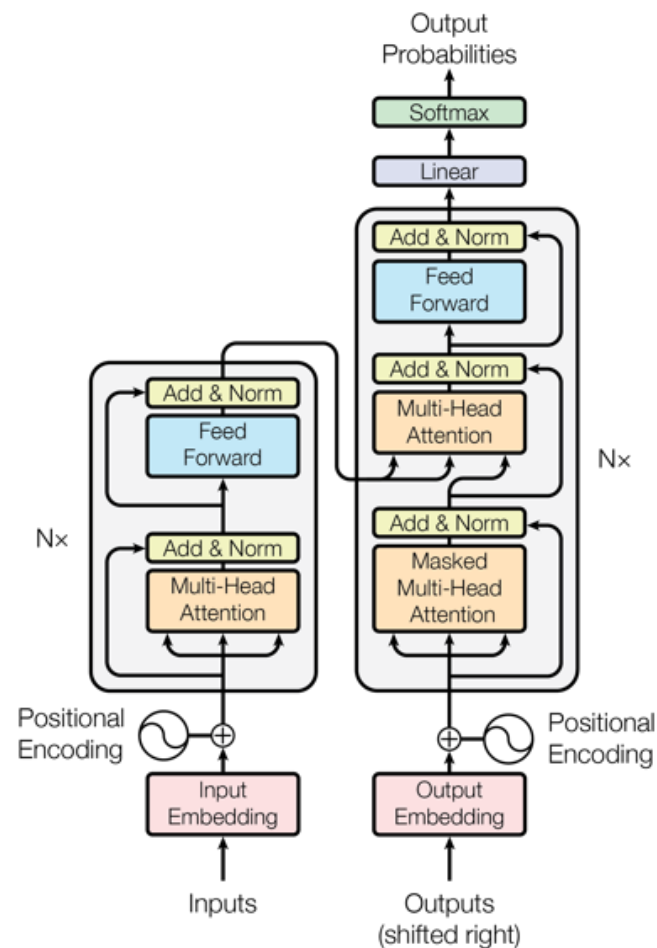
- Became popular with the paper *Attention Is All You Need* in 2017
- Introduced the *Transformer Architecture*

Attention Is All You Need

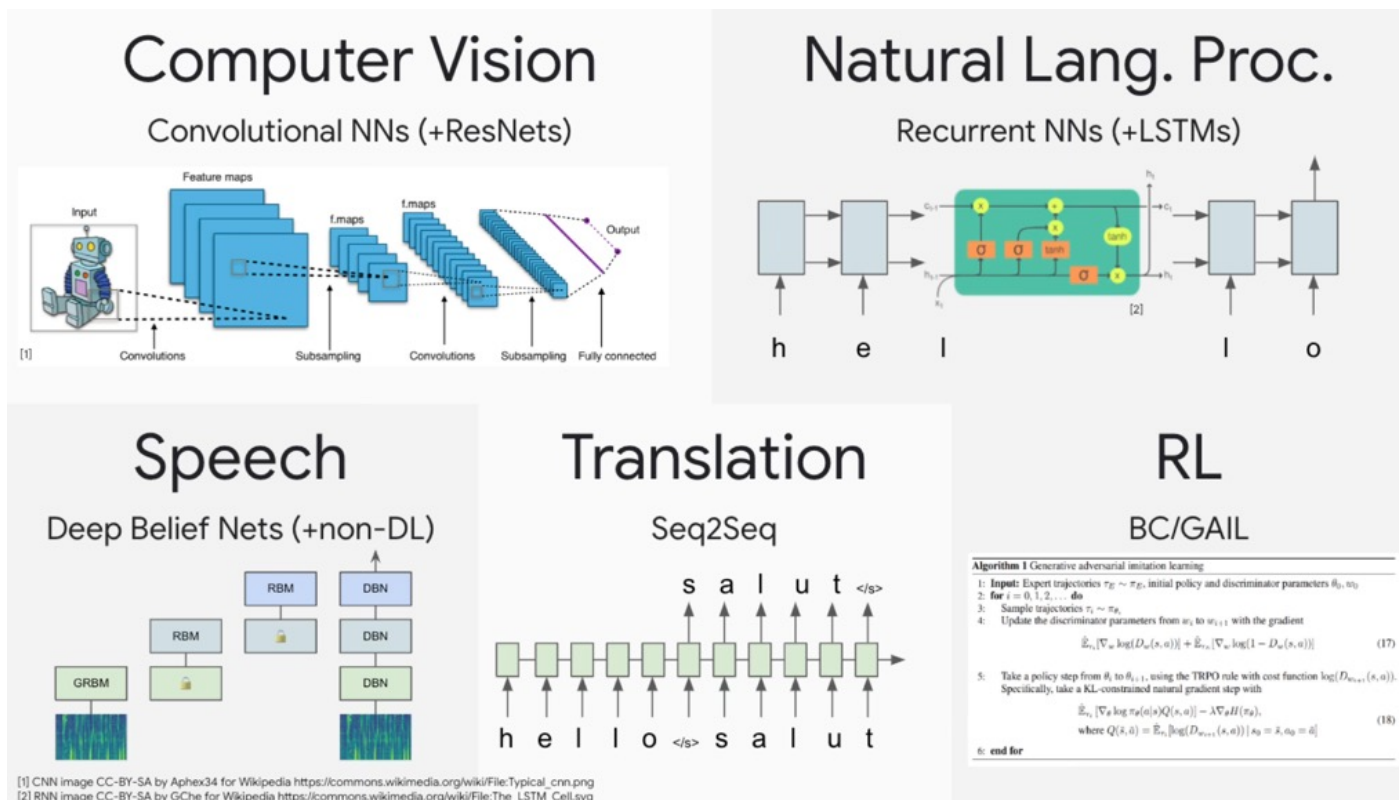
Ashish Vaswani* Google Brain avaswani@google.com	Noam Shazeer* Google Brain noam@google.com	Niki Parmar* Google Research nikip@google.com	Jakob Uszkoreit* Google Research usz@google.com
---	---	--	--

Llion Jones* Google Research llion@google.com	Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu	Łukasz Kaiser* Google Brain lukaszkaizer@google.com
--	---	--

Illia Polosukhin* †
illia.polosukhin@gmail.com



Classic Deep Learning Landscape: one architecture per 'community'



Borrowed from Lukas Beyer - Google

https://docs.google.com/presentation/d/1ZXFIhYczos679r70Yu8vV9uO6B1J0ztzeDxbnBxD1S0/edit#slide=id.g31364026ad_3_2

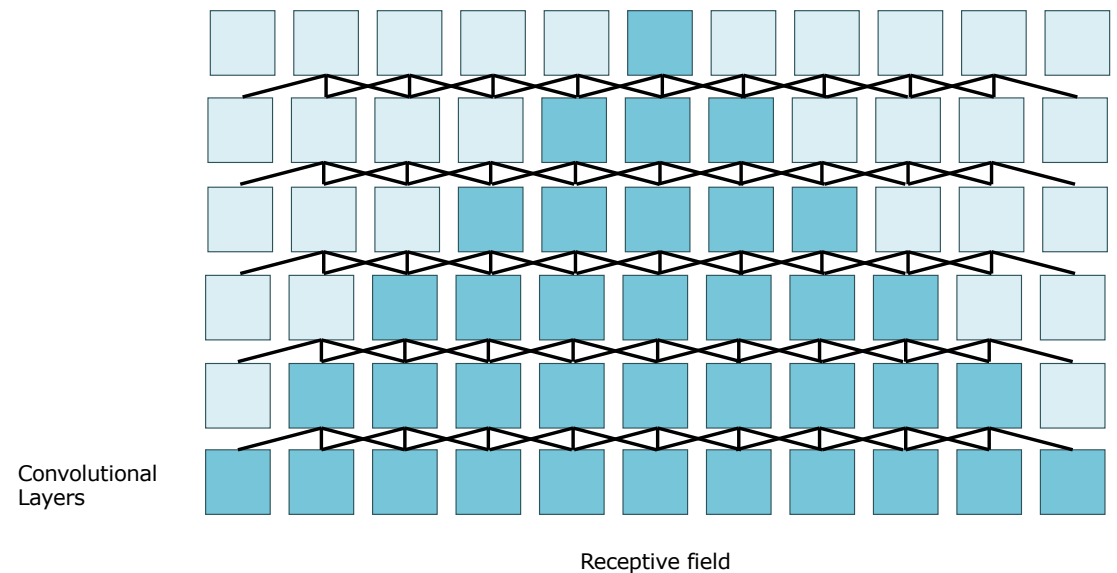
Transformer Takeover: One 'community' at a time



Transformer image source: 'Attention is All You Need' paper

CNNs

- In CNNs, information from a location travels only slowly to other locations
- This can be sped up by decreasing the image size
- But information from the edges of the image, only reach the center by the last layers of the network
- Furthermore, the information is transformed with each layer, mixing spatial locations and channels



“Attention” Mechanisms

Idea: Allow models to learn **alignments between different modalities**

for example:

- between image objects and agent actions (dynamic control problem),
- between speech frames and text (NLP),
- between pictures and text (image captioning),
- between inputs and outputs (translation),
- between different tokens in the input (self attention, NLP),
- between different image patches (self attention, image classification)
- between a (learned) key and values

Simple RNN Language Model / RNN Classification

Output distribution

$$\hat{\mathbf{y}}^{(t)} = \text{softmax} \left(\mathbf{U} \mathbf{h}^{(t)} + \mathbf{b}_2 \right) \in \mathbb{R}^{|V|}$$

Hidden states

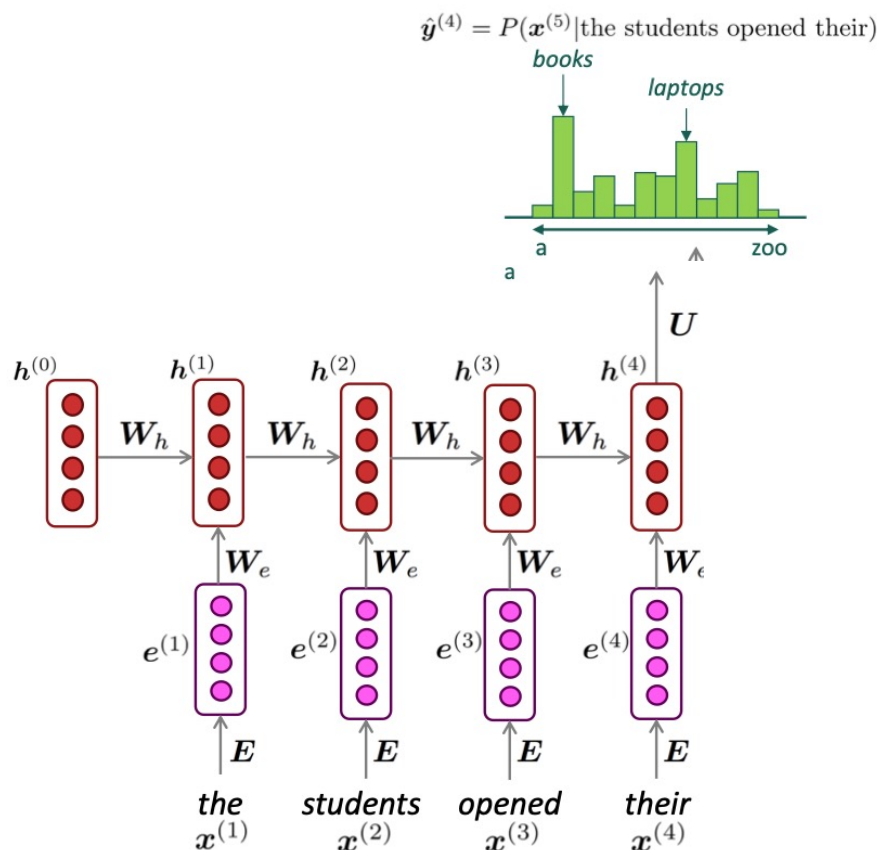
$$\mathbf{h}^{(t)} = \sigma \left(\mathbf{W}_h \mathbf{h}^{(t-1)} + \mathbf{W}_e \mathbf{e}^{(t)} + \mathbf{b}_1 \right)$$

$\mathbf{h}^{(0)}$ is the initial hidden state

Word Embeddings

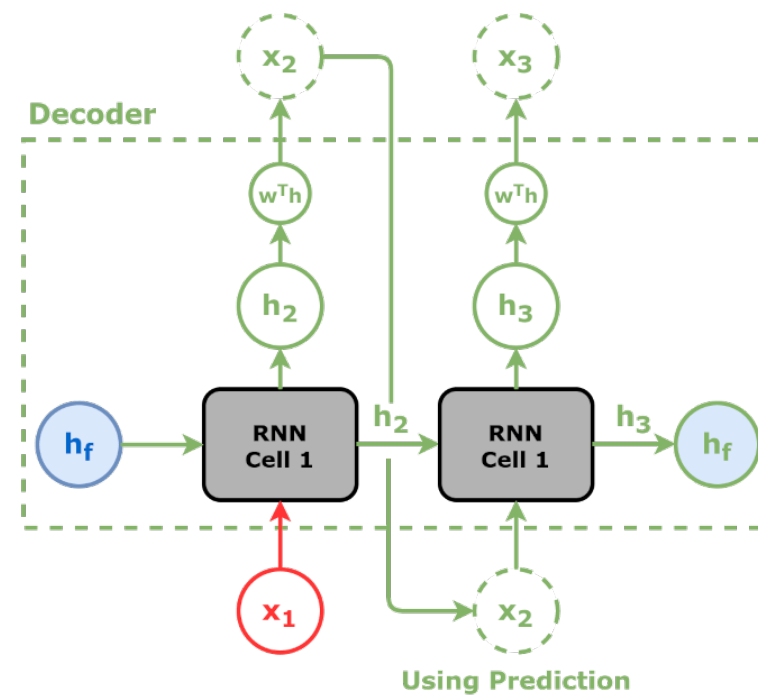
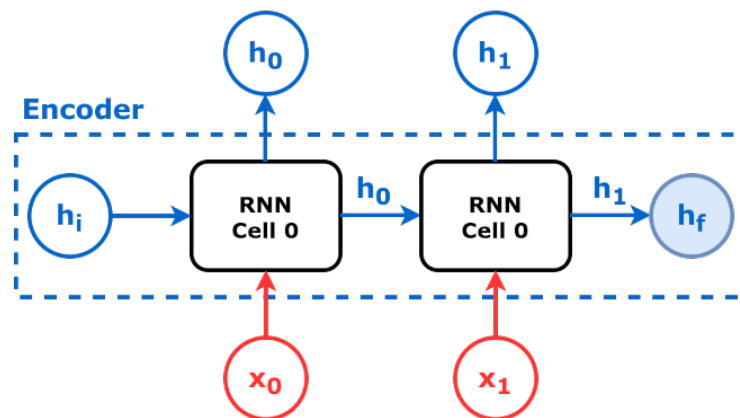
$$\mathbf{e}^{(t)} = \mathbf{E} \mathbf{x}^{(t)}$$

$$\mathbf{x}^{(t)} \in \mathbb{R}^{|V|}$$



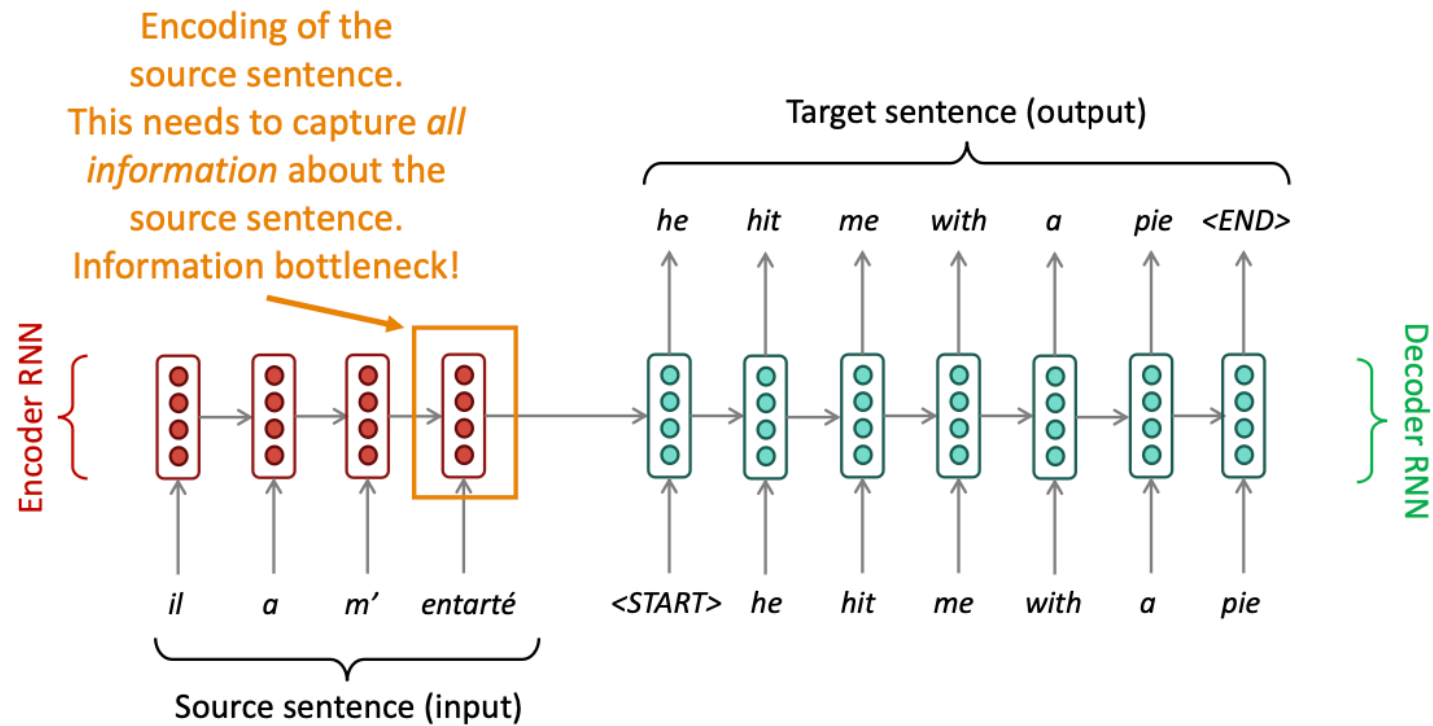
seq2seq

Sequence to sequence processing additionally uses a **decoder** receiving the encoded source state and its previous outputs

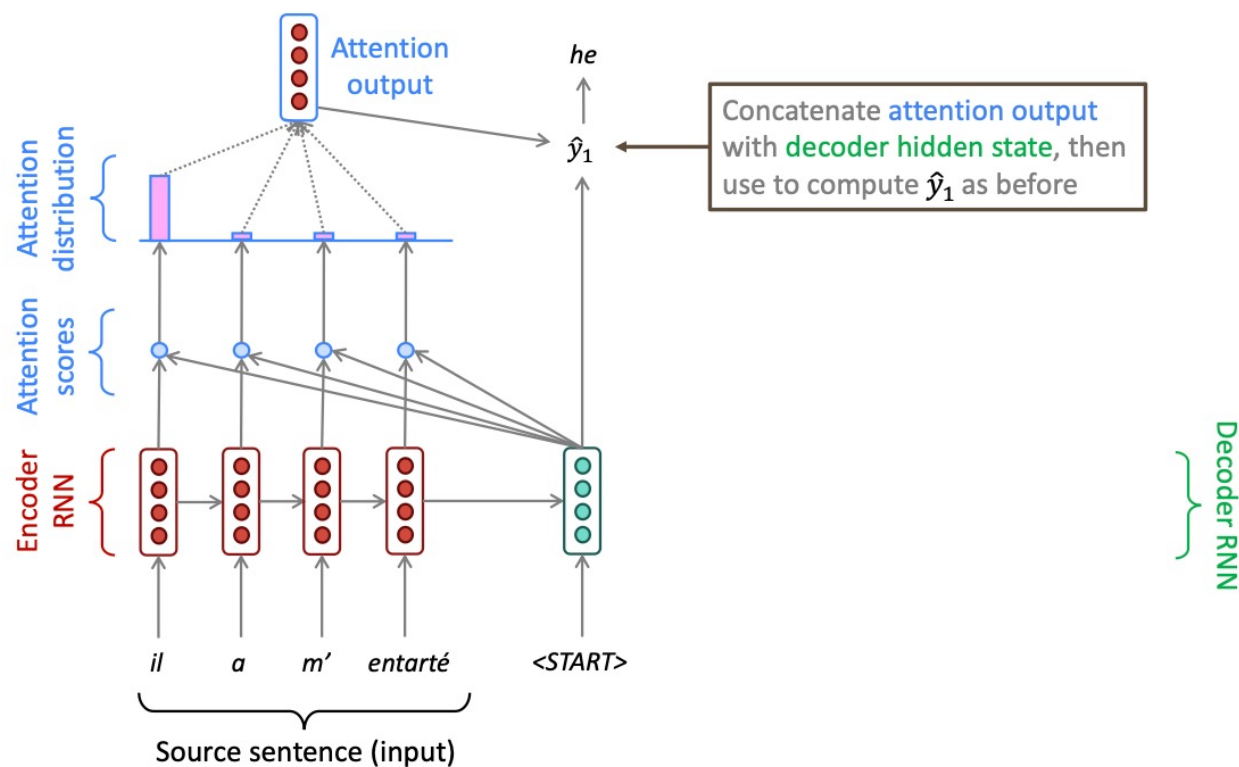


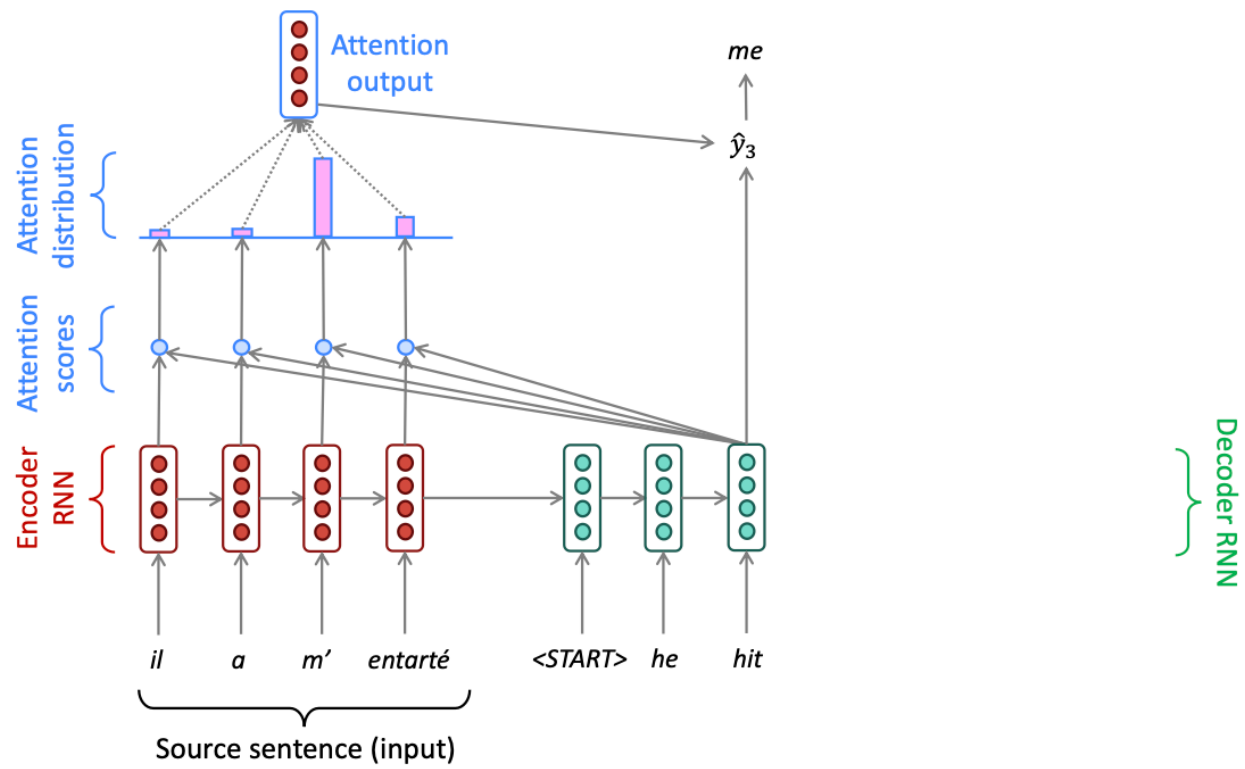
By dvgodoy - <https://dvgodoy.github.io/dl-visuals/Encoder/>, CC BY 4.0, <https://commons.wikimedia.org/w/index.php?curid=151212222>

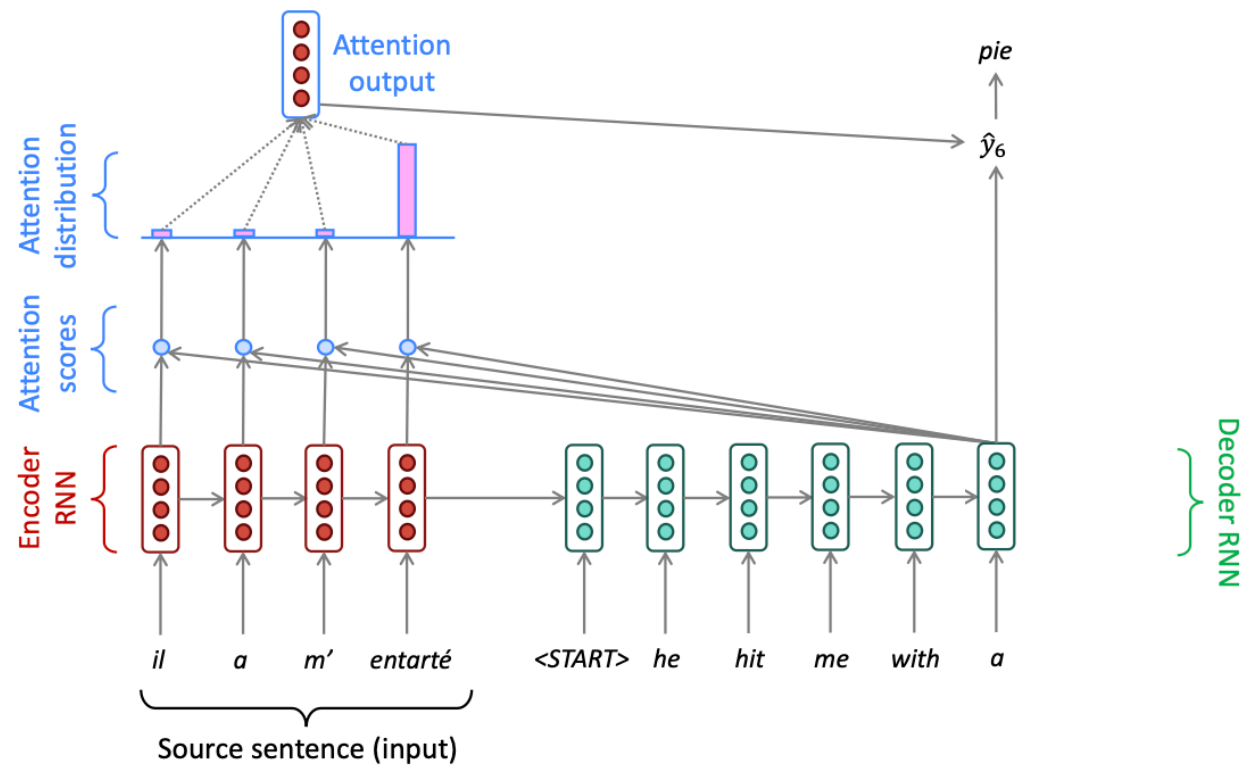
seq2seq problem



Attention mechanism allows the decoder to look back at the source







Attention: Equations

Encoder hidden states:

$$h_1, \dots, h_N \in \mathbb{R}^h$$

Decoder hidden state at t:

$$s_t \in \mathbb{R}^h$$

Attention scores:

$$\mathbf{e}^t = [s_t^T \mathbf{h}_1, \dots, s_t^T \mathbf{h}_N] \in \mathbb{R}^N$$

$$\alpha^t = \text{softmax}(\mathbf{e}^t) \in \mathbb{R}^N$$

Weighted sum of the encoder hidden states:

$$\mathbf{a}_t = \sum_{i=1}^N \alpha_i^t \mathbf{h}_i \in \mathbb{R}^h$$

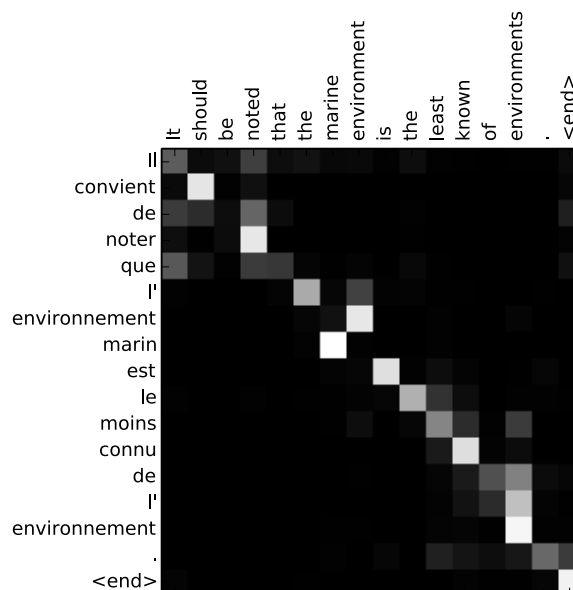
Concatenate attention output with the decoder hidden state
(and proceed as in the non-attention seq2seq model)

$$[\mathbf{a}_t; \mathbf{s}_t] \in \mathbb{R}^{2h}$$

Align and Translate

The alignment model a scores how well the inputs around position j and the outputs around positions i match.

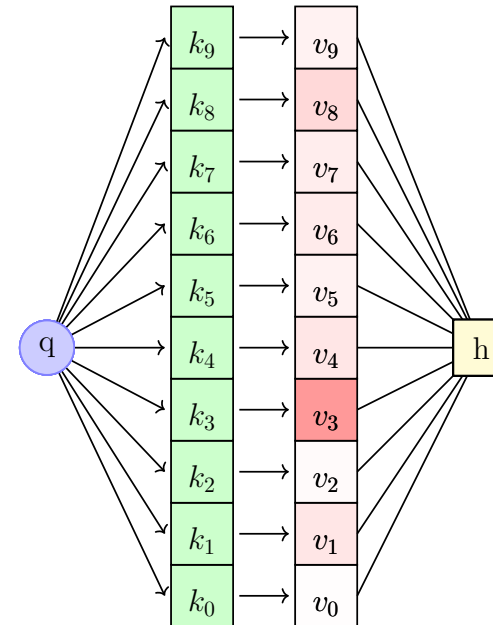
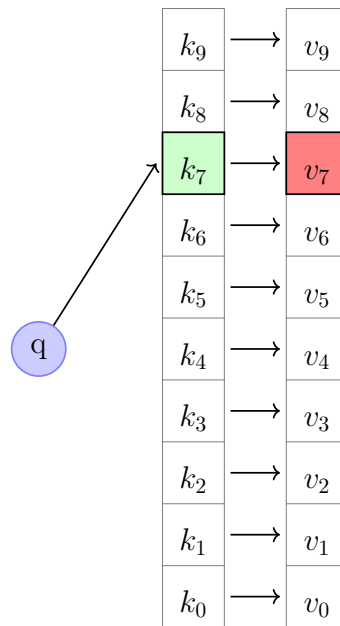
The model is parametrized as a feedforward neural network which is jointly trained with the other components of the system



$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})}$$

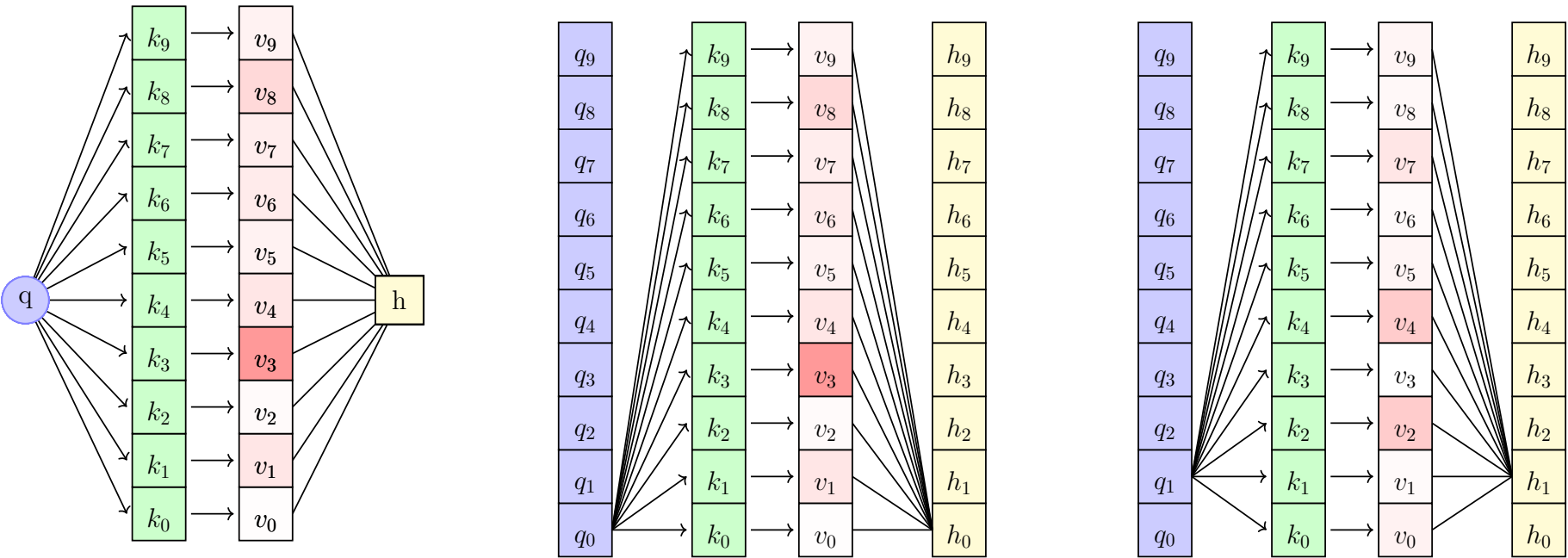
Attention Mechanism

Attention can be compared to the lookup in a hashtable where a value v is retrieved from a storage indexed by keys k with a query q

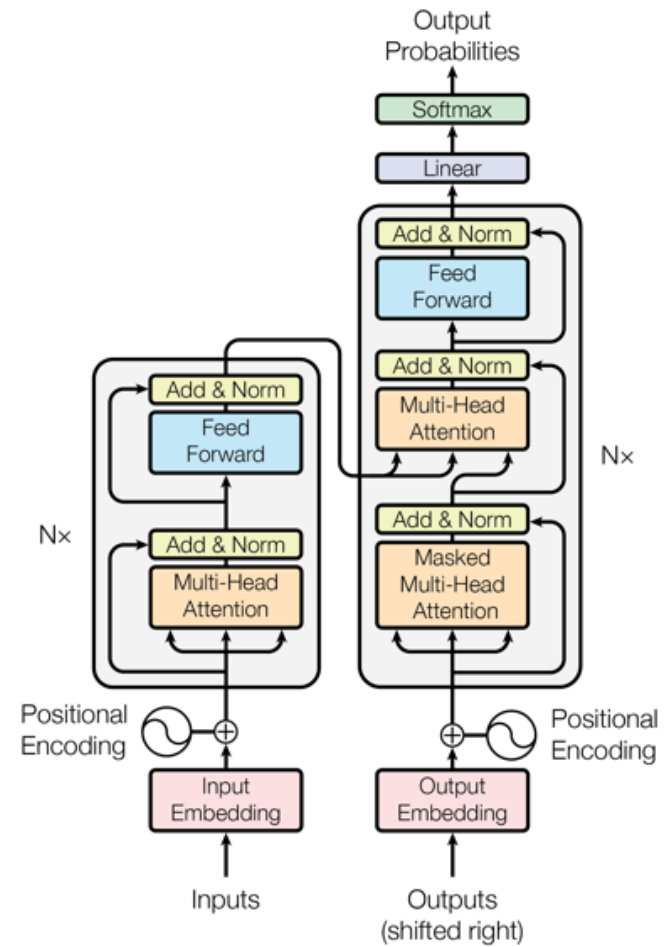


Self-Attention

- Self-Attention calculates the attention of every query



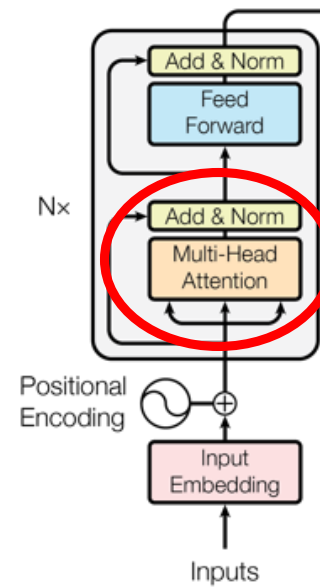
Transformers



Encoder Architecture

Self-Attention is the core building block of the Transformer model!

This is multi-headed
Self-Attention



(Vectorized) Self-Attention in the Encoder

- With embeddings stacked in X , calculate queries, keys and values:
- Calculate attention scores
- Calculate softmax
- Calculate weighted sum of the values and scale

$$Q = XW^Q \quad K = XW^K \quad V = XW^V$$

$$E = QK^T$$

$$A = \text{softmax}(E)$$

$$\text{Output} = \text{softmax} \left(QK^T / \sqrt{d_k} \right) V$$

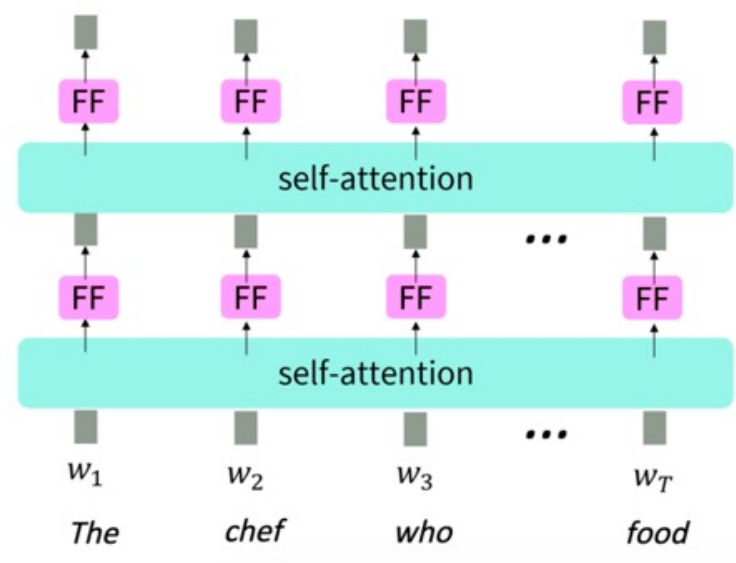
Attention is (not quite) all you need

- Self attention is simply re-averaging the value vectors
- There are no element-wise non-linearities!

Fix:

- Apply feed-forward layer (MLP) to the output of the attention

$$\begin{aligned} m_i &= \text{MLP}(\text{output}_i) \\ &= W_2 * \text{ReLU}(W_1 \times \text{output}_i + b_1) + b_2 \end{aligned}$$

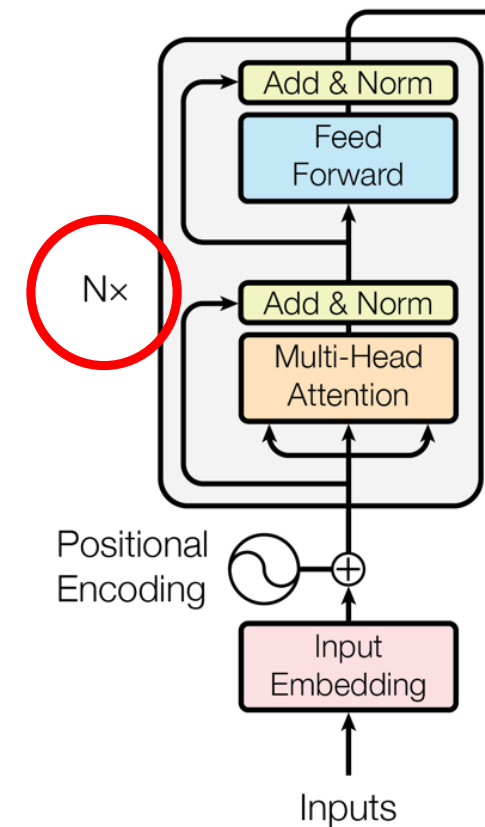


Need to go deeper...

- We would like to make these operations deeper by repeatedly stacking attention and feedforward layers.
- How to make it work for training?

Three tricks:

- Residual connections
- Layer Normalization
- Scaled Dot Product Attention

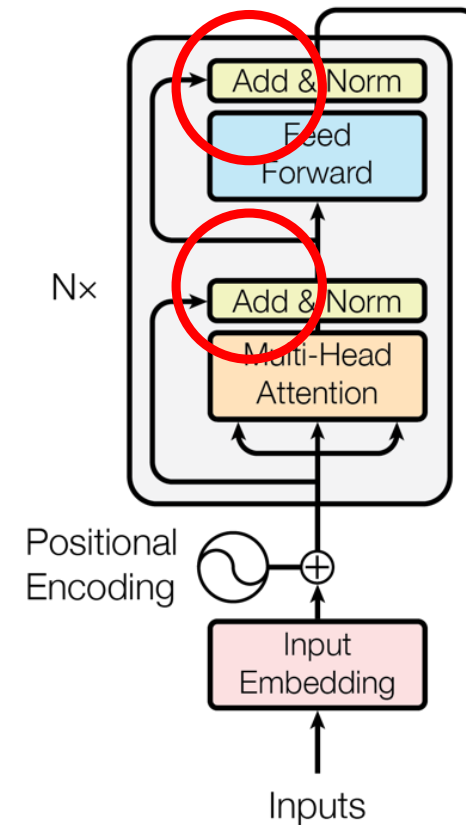


Residual connections

- Residual connections are a simple but powerful technique from computer vision (see last lecture)

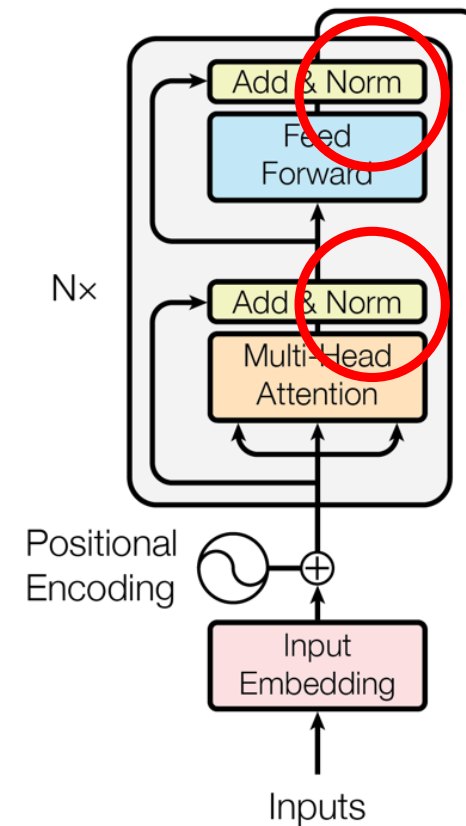
$$x_{\ell} = F(x_{\ell-1}) + x_{\ell-1}$$

- This prevents the network from "forgetting" or distorting important information as it is processed by many layers.



Layer Normalization

- **Problem:** Difficult to train the parameters of a given layer because its input from the layer beneath keeps shifting, it is not normalized
- **Solution:** Reduce variation by normalizing to zero mean and standard deviation of one within each layer.
- See batch normalization (last lecture)



Scaled Dot Product Attention

- After layer normalization, the mean and variance of the elements are 0 and 1
- The dot product still tends to take on extreme values as its variance scales with the dimensionality d_k
- (Variance of the sum = Sum of variance = d_k)
- To make the variance equal to 1, use:

$$\text{Output} = \text{softmax} \left(QK^T / \sqrt{d_k} \right) V$$

Problem: Ordering

- The output is a weighted sum of the value vectors:

$$\text{Output} = \text{softmax} \left(QK^T / \sqrt{d_k} \right) V$$

- There for the order of elements does not matter for the calculation
- Does the order of words matter in languages 😊?

He said, "I love you." (Nice thought.)

Only he said, "I love you." (No one else said it.)

He **only** said, "I love you." (He said nothing else.)

He said, "**Only** I love you." (No one else does.)

He said, "I love **only** you." (He doesn't love any one else.)

He said, "I love you **only**." (His love is exclusive.)

Position Encoding

Idea: We need to add an encoding of the position of each element i of the sequence

Consider representing each sequence index as a **position vector**

$$p_i \in \mathbb{R}^d, \text{ for } i \in \{1, 2, \dots, T\}$$

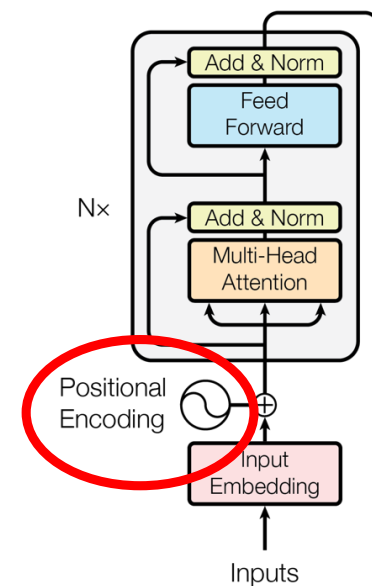
Add the position vectors to the old values, keys and queries

$$v_i = \tilde{v}_i + p_i$$

$$q_i = \tilde{q}_i + p_i$$

$$k_i = \tilde{k}_i + p_i$$

(There is some discussion of the advantages/disadvantages of adding the positional encoding vs. concatenating them to the word embeddings)



Positional Representation Through Sinusoids

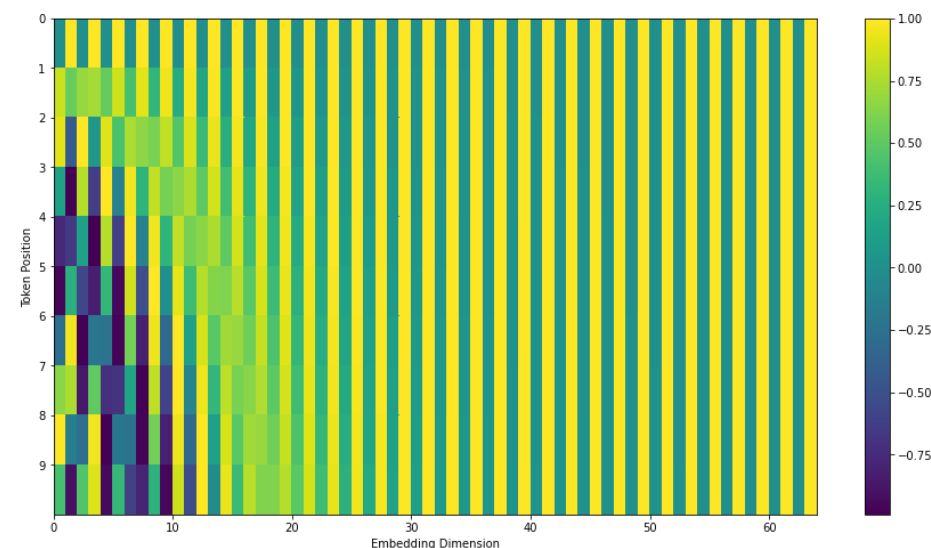
Pros:

- Periodicity indicates that maybe “absolute position” isn’t as important
- Can extrapolate to longer sequences as periods restart

Cons:

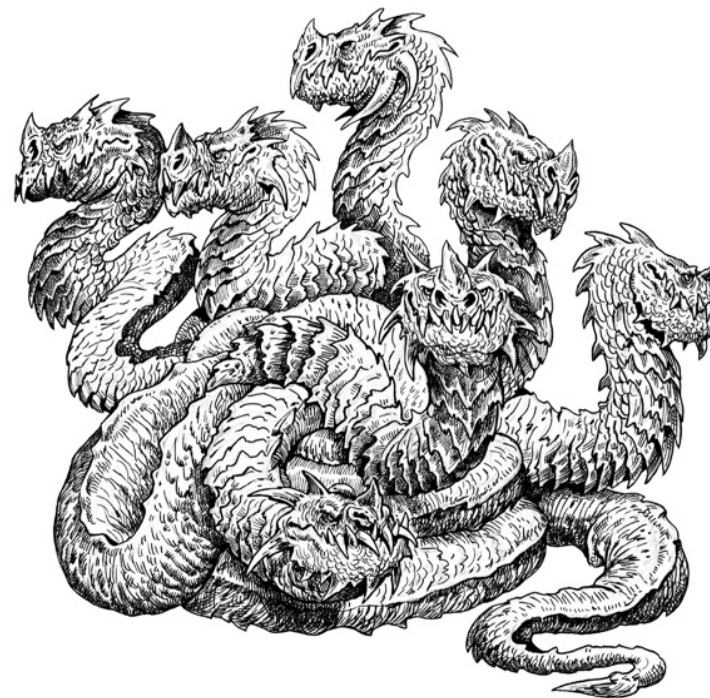
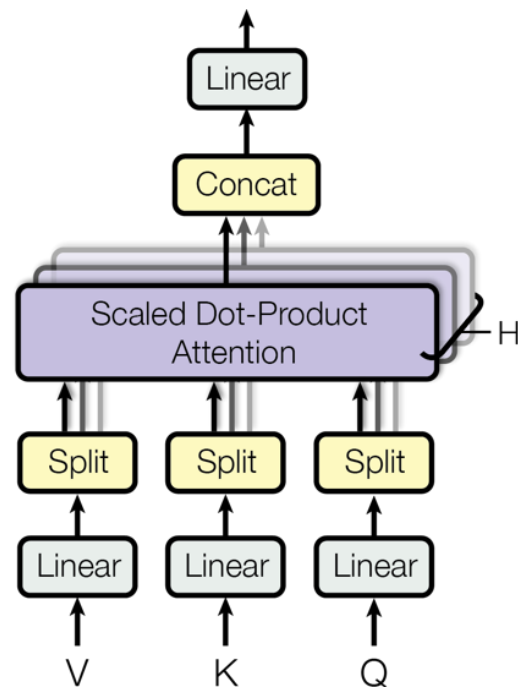
- Not learnable

$$\text{PE}_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$
$$\text{PE}_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$



Multi-Headed Self-Attention: k Heads are better than 1

Multiple heads



Multi-headed Self-Attention

- For a word at position i , self-attention *looks* where

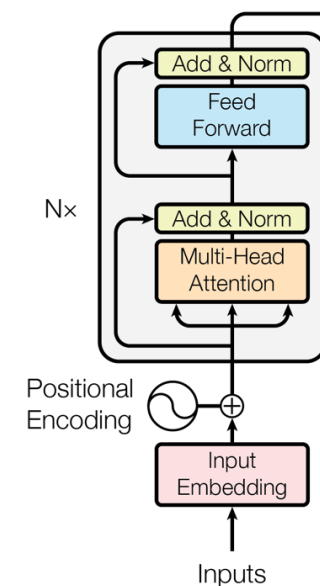
$$x_i^T Q^T K x_j$$

is high

- What if we want to focus on different positions for different reasons?
- Define **multiple attention heads** using multiple Q , K , V matrices
- Each attention head calculates attention independently
- Outputs of all the heads are combined

$$\text{output} = Y [\text{output}_1; \dots; \text{output}_h], \text{ where } Y \in \mathbb{R}^{d \times d}$$

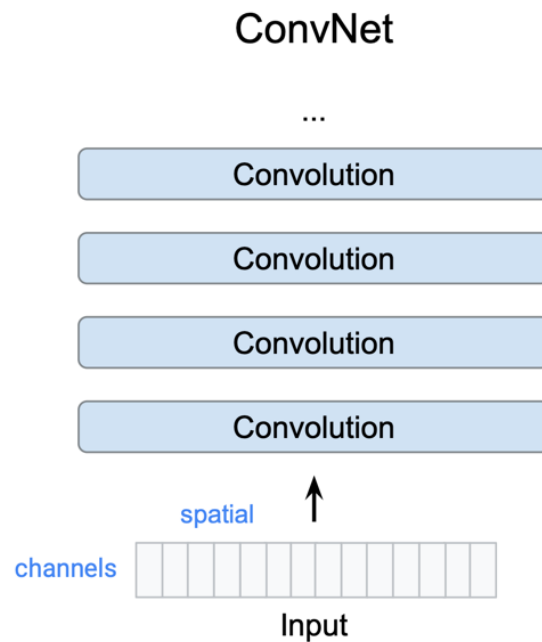
- Multiply output with a trained weight matrix, mapping the result to the length of one output



Example

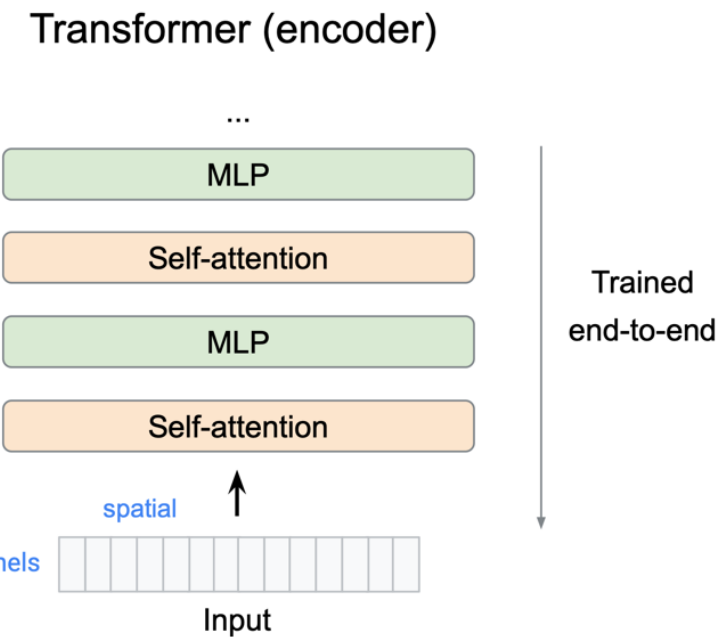
Attention and Transformer Dimensions

ConvNets vs. Transformers



Convolutions (with kernels $> 1 \times 1$) mix both the channels and the spatial locations

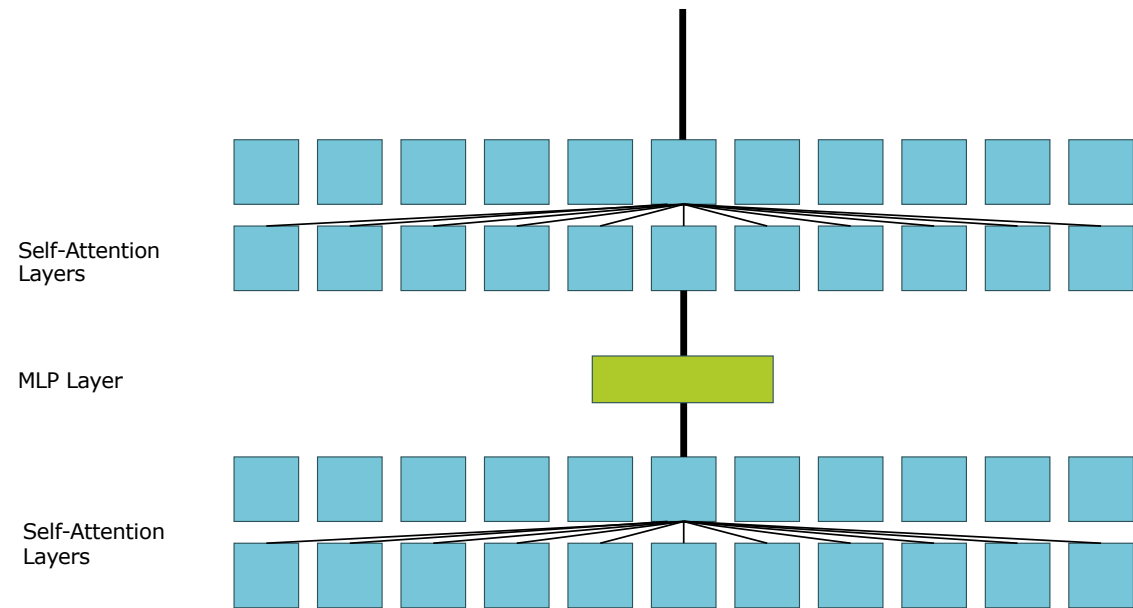
*ResNets have grouped of 1×1 convolutions that are nearly identical to transformer's MLPs



MLPs ($= 1 \times 1$ convs) only mix the channels, per location
Self-attention mixes the spatial locations (and channels a bit)

Self Attention and Transformer Architecture

- Self-Attention layers allow to quickly access information from all of the **spatial** locations
- Then an MLP transforms the **channel information**



Vision Transformer

Question:

- Can a standard Transformer be applied to image classification tasks?

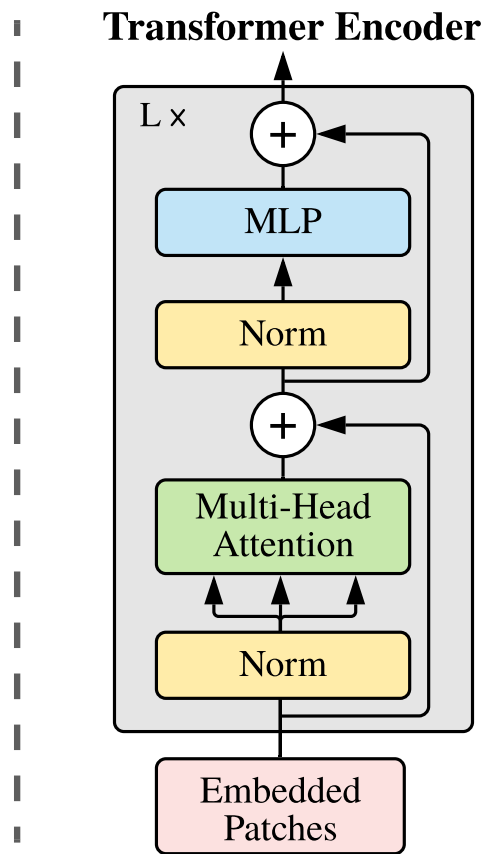
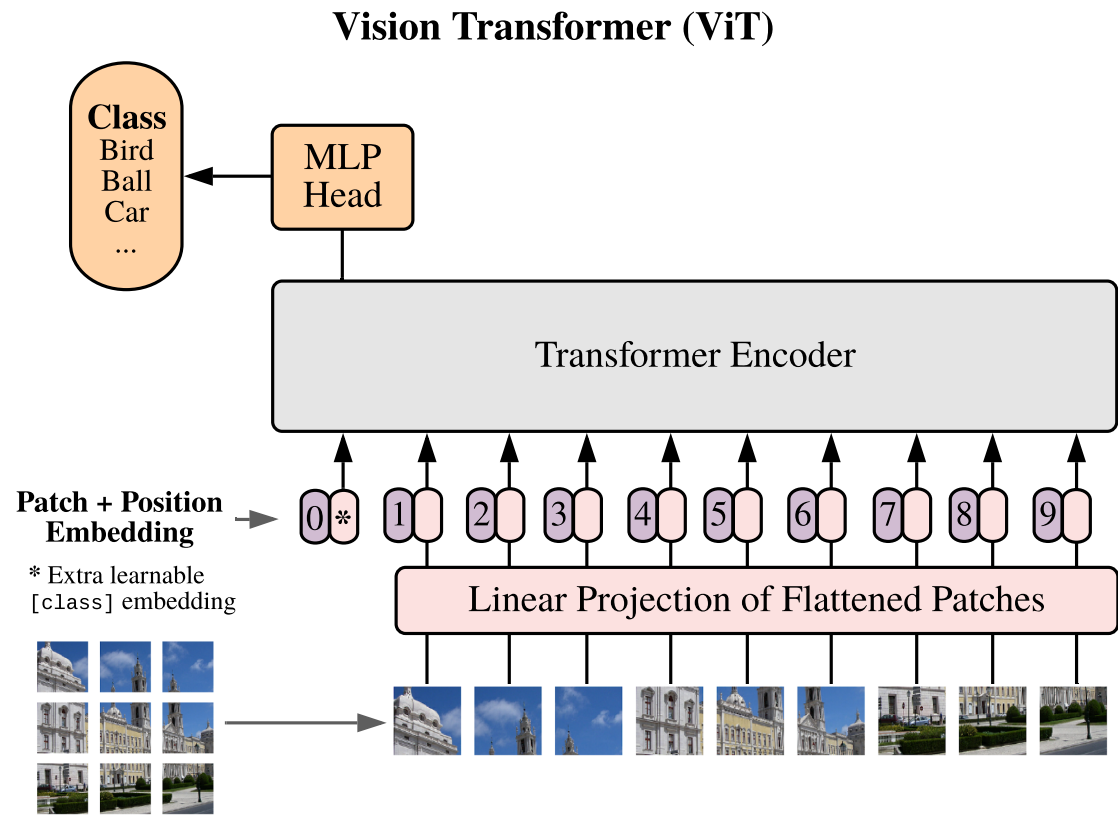
Idea:

- Split image into patches
- Calculate linear embeddings from the patches
- Use sequence of embeddings as input to a Transformer
- Train for supervised image classification

Details

- Extract square Image Patches
- Flatten Patches and project them to the embedding dimension D
- Prepend a (learnable) embedding, it's output will serve as the output of the Transformer Encoder and input to the classification head
- Classification is implemented as a MLP with one hidden layer (pre-training) or a single linear layer (fine-tuning)
- Position embeddings (1D) are added to the input embeddings

Vision Transformer (ViT)

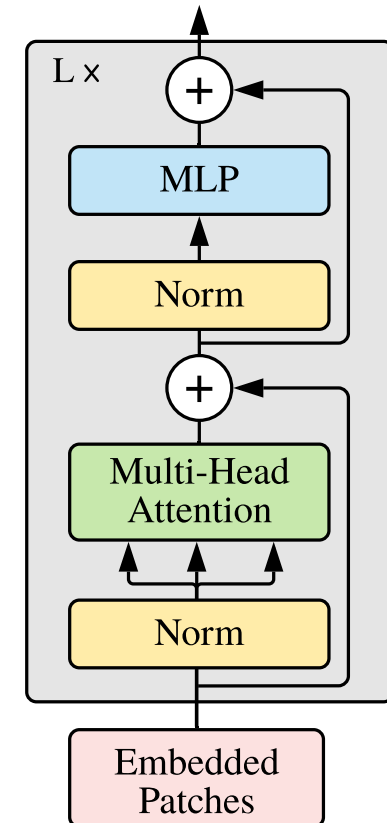


Variants

Different models with different number of layers in the Encoder and different encoding dimensions (hidden size D)

Model	Layers	Hidden size D	MLP size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

Transformer Encoder



Vision Transformer

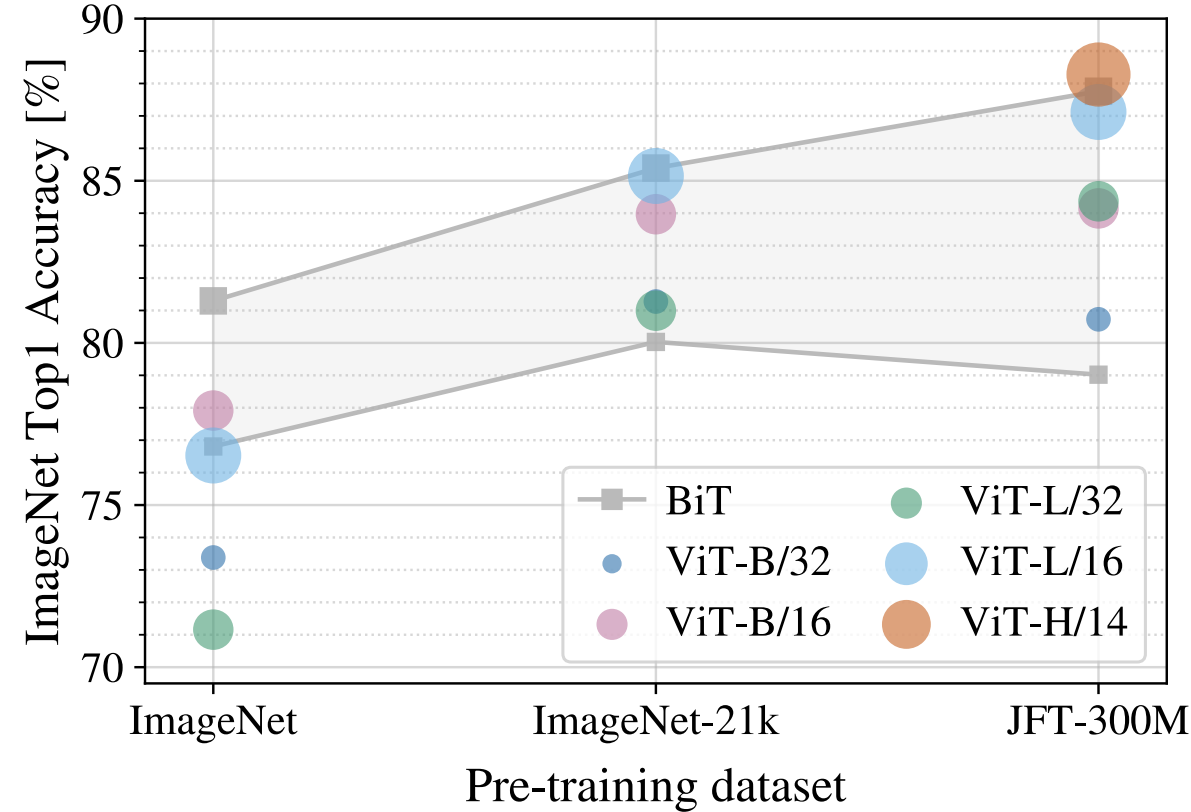
- Modest accuracies when applied to mid-sized data sets (ImageNet)
- Excellent results when pretrained on larger datasets (14M-300M) images and fine-tuned
- (Results similar to results in language models: large data set for pre-training needed)

Results

Results on Image classification tasks, all models use pre-training, either on ImageNet 21k or JFT (internal google) data sets

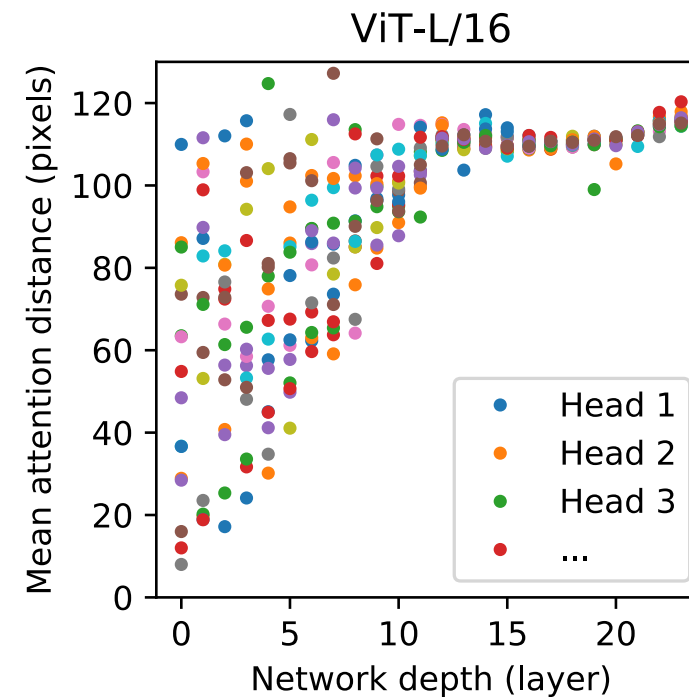
	Ours-JFT (ViT-H/14)	Ours-JFT (ViT-L/16)	Ours-I21k (ViT-L/16)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
ImageNet	88.55 ± 0.04	87.76 ± 0.03	85.30 ± 0.02	87.54 ± 0.02	88.4/88.5*
ImageNet ReaL	90.72 ± 0.05	90.54 ± 0.03	88.62 ± 0.05	90.54	90.55
CIFAR-10	99.50 ± 0.06	99.42 ± 0.03	99.15 ± 0.03	99.37 ± 0.06	—
CIFAR-100	94.55 ± 0.04	93.90 ± 0.05	93.25 ± 0.05	93.51 ± 0.08	—
Oxford-IIIT Pets	97.56 ± 0.03	97.32 ± 0.11	94.67 ± 0.15	96.62 ± 0.23	—
Oxford Flowers-102	99.68 ± 0.02	99.74 ± 0.00	99.61 ± 0.02	99.63 ± 0.03	—
VTAB (19 tasks)	77.63 ± 0.23	76.28 ± 0.46	72.72 ± 0.21	76.29 ± 1.70	—
TPUv3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k

Effect on data set size



Mean attention distance

- Attention distance can be compared to the receptive fields of CNNs
- Some attention heads use large distances, i.e. look at image patches that are far away already in the lower layers

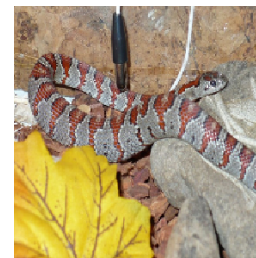
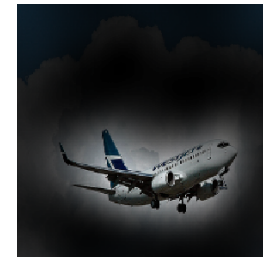
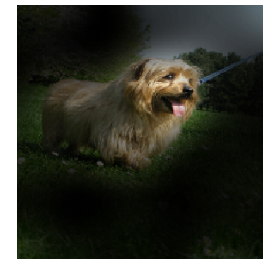
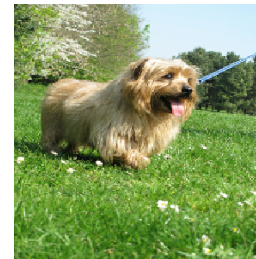


Attention Visualization

Attention weights show that the transformer concentrates on relevant parts of the image

Input

Attention



Exercise

- Implementation of Visual Transformer in torch
- Use TransformerEncoderLayers from torch

Can CNNs and Attention be combined?

First example from the Visual Transformer Paper:

Hybrid Architecture. As an alternative to raw image patches, the input sequence can be formed from feature maps of a CNN (LeCun et al., 1989). In this hybrid model, the patch embedding projection $\mathbf{E}$ (Eq. 1) is applied to patches extracted from a CNN feature map. As a special case, the patches can have spatial size 1x1, which means that the input sequence is obtained by simply flattening the spatial dimensions of the feature map and projecting to the Transformer dimension. The classification input embedding and position embeddings are added as described above.

- Are there other possibilities to combine CNNs and Attention?
- Is it beneficial?

CoAtNet

- Transformers in computer vision still fall behind SOTA CNN approaches on many tasks, especially when only low data amounts are available
- Transformers tend to have larger model capacity, but their generalization can be worse (lack of inductive bias)

Idea

- Combine Convolution and Attention
- There have been different attempts to that, the question is **how** the combination is done most effectively and how to balance the trade-offs between **accuracy** and **efficiency**

Convolution and Attention Calculations

- In some of the most successful CNNs, depthwise convolution is used (see first lecture), this can be written as

$$y_i = \sum_{j \in \mathcal{L}(i)} w_{i-j} \odot x_j \quad (\text{depthwise convolution})$$

- (where $\mathcal{L}(i)$ is a neighborhood of i , i.e. 3×3)
- Self-attention can be written as

$$y_i = \sum_{j \in \mathcal{G}} \frac{\exp(x_i^\top x_j)}{\underbrace{\sum_{k \in \mathcal{G}} \exp(x_i^\top x_k)}_{A_{i,j}}} x_j \quad (\text{self-attention}),$$

Comparison

- The kernel w is **input-independent**, attention weights depend on the input dynamically. It is easier for attention to capture complicated relational interactions between spatial positions
- For (i,j) the kernel weight only depends on their relative position (translational invariance), this improves generalization. Transformers use absolute positioning.
- Attention has a global receptive field which provides more contextual information, but requires significantly more computation.

Combination of CNN and Attention ideas

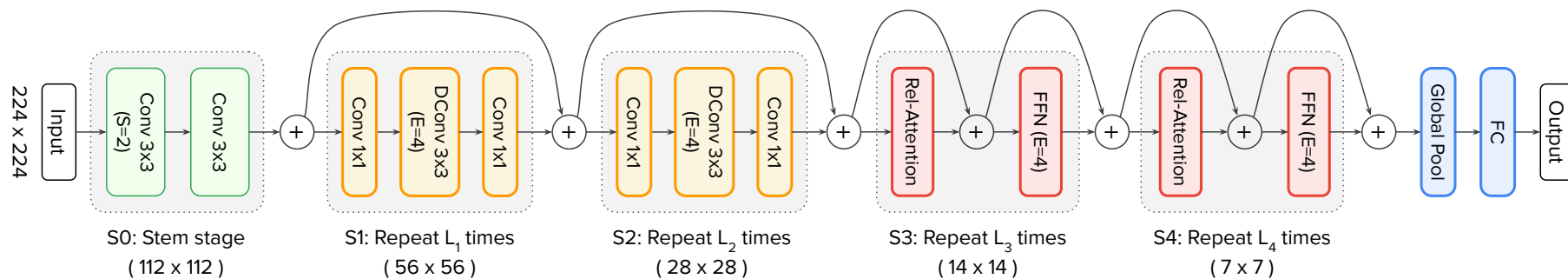
- Calculate the attention weights using the input adaptive $x_i^\top x_j$ value and the global kernel value w_{i-j}

$$y_i^{\text{pre}} = \sum_{j \in \mathcal{G}} \frac{\exp(x_i^\top x_j + w_{i-j})}{\sum_{k \in \mathcal{G}} \exp(x_i^\top x_k + w_{i-k})} x_j$$

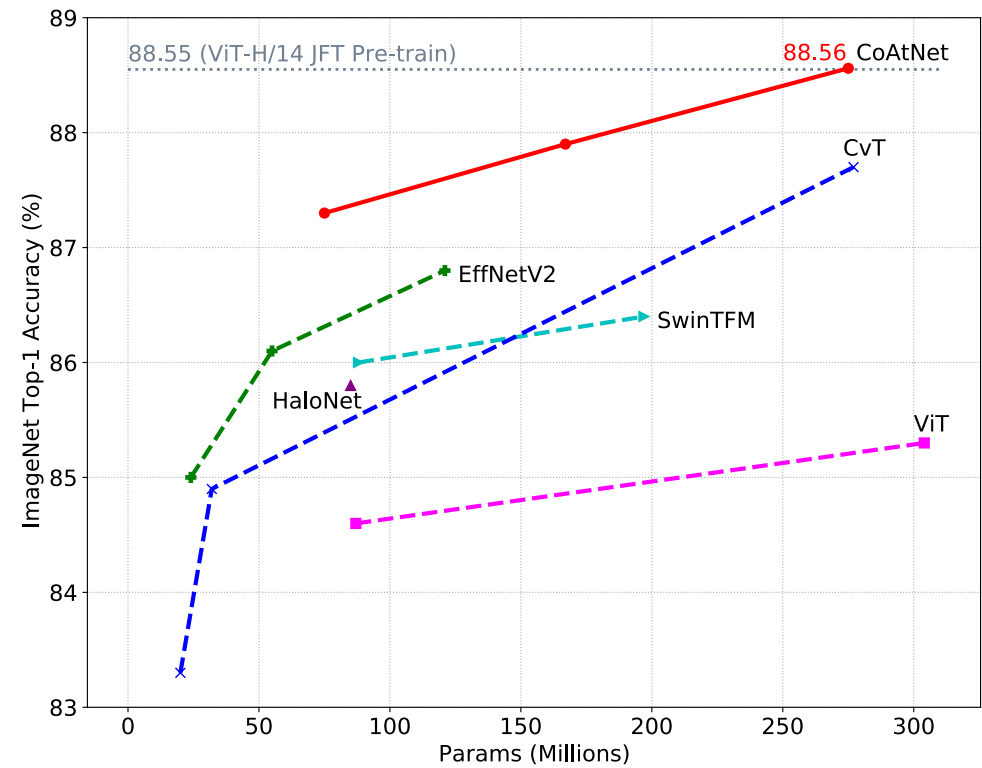
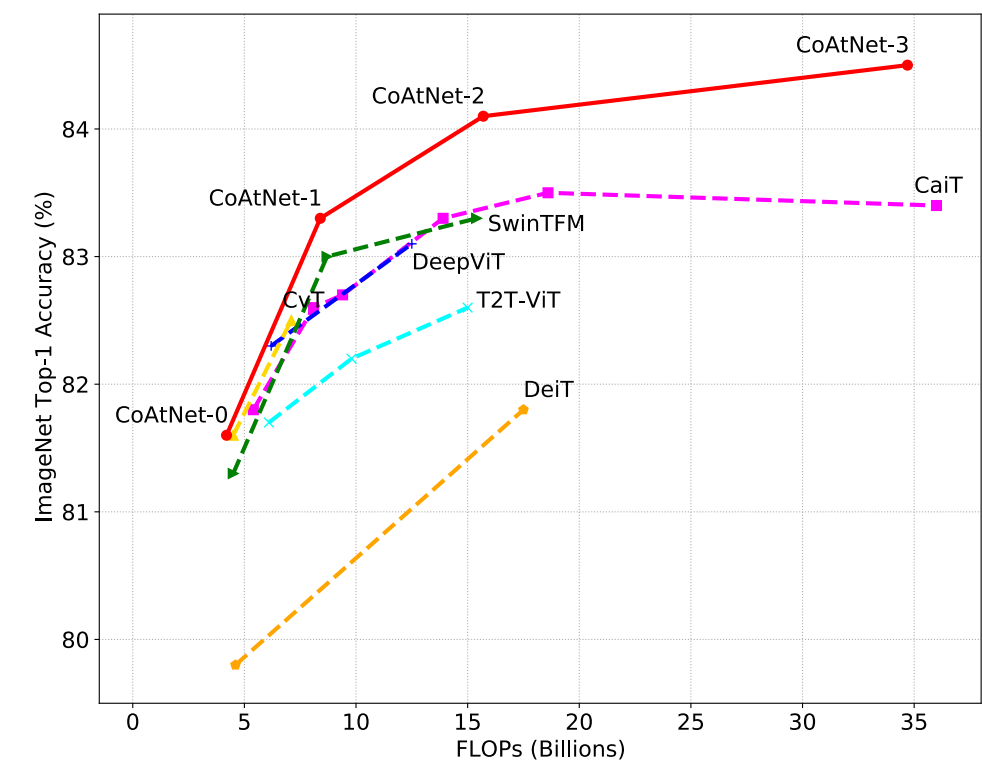
- This is used as a modified transformer block in the CoAtNet Model, it is also known as *relative self-attention*

Vertical Design

- Applying the relative-attention at pixel level is computationally not possible
- A down-sampling of the image is needed (for example with strided convolutions)
- Mimic CNNs architecture: use different stages and down-sample at the beginning of each stage while increasing the number of channels
- Tests showed best results (for generalization, model capacity and transferability) with 3 convolution blocks, followed by 2 transformer blocks



Results



Comparison of accuracy vs FLOPs and Accuracy vs. Nr Parameters

**Lucerne University of
Applied Sciences and Arts
Information Technology**

Research

Prof. Dr. Thomas Koller

Lecturer

Phone direct +41 41 757 68 32
thomas.koller@hslu.ch